ALLIANCE

UNIVERSITY

Private University established in Karnataka State by Act No. 34 of year 2010
Recognised by the University Grants Commission (UGC), New Delhi

DATABASE MANAGEMENT SYSTEM

CAB BOOKING MANAGEMENT SYSTEM

ASSIGNMENT 5

DATABASE CREATION AND TABLE IMPLEMENTATION

Submitted by: ***Preksha V***
Register Number: ***2511021240063***
Programme: Bachelors of Computer Application - III

Submitted to: ***Renita Joy R***
Subject: Database Management System (DBMS)

SQL DDL implementation of the database designed in Assignments 1, 2 and 3.

1. Introduction

This assignment implements the Cab Booking Management System database using SQL Data Definition Language (DDL) commands. The implementation follows the finalized design from the earlier assignments: the eight entities and relationships from Assignments 1 and 2, with the improvements recommended in Assignment 3 (UNIQUE constraints for one-to-one relationships, CHECK constraints for valid data, and removal of the duplicated Customer_ID and Driver_ID from RATING).

The commands are written for MySQL 8.0 or later and are kept simple: CREATE DATABASE, CREATE TABLE, PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT and CHECK.

2. Creating the Database

The database is created first and then selected so that all tables are created inside it.

```
CREATE DATABASE CabBookingDB;
USE CabBookingDB;
```

3. Table Creation Order

Tables that are referenced by foreign keys (parent tables) must exist before the tables that refer to them (child tables). The tables are therefore created in the following order.

Order	Table	Type	Depends on
1	CUSTOMER	Parent	-
2	DRIVER	Parent	-
3	CAB	Child	DRIVER
4	BOOKING	Child	CUSTOMER, DRIVER, CAB
5	TRIP	Child	BOOKING
6	PAYMENT	Child	BOOKING
7	RATING	Child	BOOKING
8	DRIVER_LOCATION	Child	DRIVER

4. SQL DDL Commands for Each Table

4.1 CUSTOMER

Stores passengers who use the application. Phone and Email are UNIQUE so the same contact is not registered twice.

```
CREATE TABLE CUSTOMER (
    Customer_ID INT PRIMARY KEY,
    Name        VARCHAR(50)  NOT NULL,
    Phone       VARCHAR(15)  NOT NULL UNIQUE,
    Email       VARCHAR(80)  UNIQUE,
    Address     VARCHAR(150)
);
```

4.2 DRIVER

Stores registered drivers. License_Number and Phone are UNIQUE; Status is restricted to Available, Busy or Offline.

```
CREATE TABLE DRIVER (
    Driver_ID      INT PRIMARY KEY,
    Name           VARCHAR(50) NOT NULL,
    Phone          VARCHAR(15) NOT NULL UNIQUE,
    License_Number VARCHAR(20) NOT NULL UNIQUE,
    Status         VARCHAR(10) NOT NULL DEFAULT 'Available',
    CHECK (Status IN ('Available', 'Busy', 'Offline'))
);
```

4.3 CAB

Stores vehicles. UNIQUE on Driver_ID keeps the Driver-Cab 1:1 rule; Vehicle_Number is UNIQUE; Cab_Type and Status are restricted to valid values.

```
CREATE TABLE CAB (
    Cab_ID         INT PRIMARY KEY,
    Driver_ID      INT UNIQUE,
    Vehicle_Number VARCHAR(15) NOT NULL UNIQUE,
    Cab_Type       VARCHAR(10) NOT NULL,
    Model          VARCHAR(30),
    Status         VARCHAR(15) NOT NULL DEFAULT 'Available',
    FOREIGN KEY (Driver_ID) REFERENCES DRIVER(Driver_ID),
    CHECK (Cab_Type IN ('Sedan', 'SUV', 'Hatchback')),
    CHECK (Status IN ('Available', 'Booked', 'Maintenance'))
);
```

4.4 BOOKING

Records each reservation with its customer, driver and cab. Three foreign keys connect it to the parent tables.

```
CREATE TABLE BOOKING (
    Booking_ID      INT PRIMARY KEY,
    Customer_ID     INT NOT NULL,
    Driver_ID       INT NOT NULL,
    Cab_ID          INT NOT NULL,
    Booking_Date    DATETIME NOT NULL,
    Pickup_Location VARCHAR(100) NOT NULL,
    Drop_Location   VARCHAR(100) NOT NULL,
    Booking_Status  VARCHAR(10) NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID),
    FOREIGN KEY (Driver_ID)   REFERENCES DRIVER(Driver_ID),
    FOREIGN KEY (Cab_ID)      REFERENCES CAB(Cab_ID),
    CHECK (Booking_Status IN ('Pending', 'Confirmed', 'Cancelled', 'Completed'))
);
```

4.5 TRIP

Stores journey details. UNIQUE on Booking_ID enforces Booking-Trip 1:1; CHECK constraints prevent negative values and an end time earlier than the start time.

```
CREATE TABLE TRIP (
    Trip_ID     INT PRIMARY KEY,
    Booking_ID  INT NOT NULL UNIQUE,
    Start_Time  DATETIME,
    End_Time    DATETIME,
    Distance    DECIMAL(6,2),
    Fare        DECIMAL(8,2),
    Trip_Status VARCHAR(10) NOT NULL DEFAULT 'Started',
    FOREIGN KEY (Booking_ID) REFERENCES BOOKING(Booking_ID),
    CHECK (Distance >= 0),
    CHECK (Fare >= 0),
    CHECK (End_Time IS NULL OR End_Time >= Start_Time),
    CHECK (Trip_Status IN ('Started', 'Completed', 'Cancelled'))
);
```

4.6 PAYMENT

Stores the payment for a booking. UNIQUE on Booking_ID enforces Booking-Payment 1:1; method and status are restricted to valid values.

```
CREATE TABLE PAYMENT (
    Payment_ID     INT PRIMARY KEY,
    Booking_ID     INT NOT NULL UNIQUE,
    Amount         DECIMAL(8,2) NOT NULL,
    Payment_Method VARCHAR(10) NOT NULL,
    Payment_Date   DATETIME,
    Payment_Status VARCHAR(10) NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (Booking_ID) REFERENCES BOOKING(Booking_ID),
    CHECK (Amount >= 0),
    CHECK (Payment_Method IN ('UPI', 'Card', 'Cash')),
    CHECK (Payment_Status IN ('Successful', 'Pending', 'Failed'))
);
```

4.7 RATING

Stores optional feedback. UNIQUE on Booking_ID enforces Booking-Rating 1:0..1; Rating must be between 1 and 5. Customer and driver are found through BOOKING.

```
CREATE TABLE RATING (
    Rating_ID  INT PRIMARY KEY,
    Booking_ID INT NOT NULL UNIQUE,
    Rating     INT NOT NULL,
    Review     VARCHAR(255),
    FOREIGN KEY (Booking_ID) REFERENCES BOOKING(Booking_ID),
    CHECK (Rating BETWEEN 1 AND 5)
);
```

4.8 DRIVER_LOCATION

Stores repeated location updates of a driver. One driver can have many records (1:M).

```
CREATE TABLE DRIVER_LOCATION (
    Location_ID  INT PRIMARY KEY,
    Driver_ID    INT NOT NULL,
    Latitude     DECIMAL(9,6) NOT NULL,
    Longitude    DECIMAL(9,6) NOT NULL,
    Updated_Time DATETIME NOT NULL,
    FOREIGN KEY (Driver_ID) REFERENCES DRIVER(Driver_ID)
);
```

5. Relationships Implemented

The relationships from Assignment 2 are implemented using foreign keys, with UNIQUE constraints where a one-to-one relationship is required.

Relationship	Cardinality	How it is implemented
Customer - Booking	1 : M	BOOKING.Customer_ID references CUSTOMER(Customer_ID)
Driver - Booking	1 : M	BOOKING.Driver_ID references DRIVER(Driver_ID)
Driver - Cab	1 : 1	CAB.Driver_ID references DRIVER(Driver_ID) and is UNIQUE
Cab - Booking	1 : M	BOOKING.Cab_ID references CAB(Cab_ID)
Booking - Trip	1 : 1	TRIP.Booking_ID references BOOKING(Booking_ID) and is UNIQUE
Booking - Payment	1 : 1	PAYMENT.Booking_ID references BOOKING(Booking_ID) and is UNIQUE
Booking - Rating	1 : 0..1	RATING.Booking_ID references BOOKING(Booking_ID) and is UNIQUE
Driver - Driver_Location	1 : M	DRIVER_LOCATION.Driver_ID references DRIVER(Driver_ID)

6. Constraints Used

Constraint	Where it is used	Purpose
PRIMARY KEY	Every table (Customer_ID, Driver_ID, Cab_ID, Booking_ID, Trip_ID, Payment_ID, Rating_ID, Location_ID)	Uniquely identifies each record
FOREIGN KEY	CAB, BOOKING, TRIP, PAYMENT, RATING, DRIVER_LOCATION	Links tables and prevents orphan records
UNIQUE	Phone, Email, License_Number, Vehicle_Number, and Booking_ID in TRIP, PAYMENT and RATING	Prevents duplicates and enforces 1:1 rules
NOT NULL	Names, phone numbers, locations, amounts and all foreign keys that must exist	Ensures required data is always entered
DEFAULT	Status columns (for example Pending, Available, Started)	Gives a sensible starting value
CHECK	Status and method lists, Rating 1 to 5, non-negative Distance, Fare and Amount, End_Time not before Start_Time	Prevents invalid or contradictory values

7. Verifying the Tables

After execution, the following commands list the tables and display the structure of each one.

```
SHOW TABLES;
DESCRIBE CUSTOMER;
DESCRIBE DRIVER;
DESCRIBE CAB;
DESCRIBE BOOKING;
DESCRIBE TRIP;
DESCRIBE PAYMENT;
DESCRIBE RATING;
DESCRIBE DRIVER_LOCATION;
```

8. Conclusion

The Cab Booking Management System database has been created using simple SQL DDL commands. All eight tables are implemented with primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT and CHECK constraints. The relationships documented in Assignment 2 are enforced by the database itself, and the Assignment 3 improvements reduce redundancy and keep the data consistent.